RC Quick Byte: Supercharge Your Python with Dask

Research Computing
UNIVERSITY OF COLORADO BOULDER

Supercharge Your Python with Dask

- Date: September 8, 2026
- Instructor: Mohal Khandelwal
- Website: www.rc.colorado.edu/rc
- Helpdesk: rc-help@colorado.edu
- Documentation: https://curc.readthedocs.io/en/latest/programming/dask.html

CURC DASK DOCS

Scan for the full walkthrough

What you'll learn

01	Why single-core Python hits a wall
02	What Dask is, and how a cluster is wired
03	Launch a cluster from Open OnDemand Jupyter Notebook
04	Dask Dashboard
05	Scale NumPy and pandas: Arrays and DataFrames
06	Live demo on Alpine, plus when Dask is worth it

Python is great… until it isn't

MEMORY

Your data doesn't fit

- Your dataset is larger than available RAM.
- Pandas and NumPy often load the data into memory at once.
- Large files can quickly lead to Memory Error.
- You need a way to process the data in smaller pieces.

CPU

One core is doing all the work

- Large computations can take hours even when other cores are idle.
- Manually splitting and coordinating the work is difficult.

SCALING

Your workflow outgrows one machine

- Traditional approaches may require significant changes to your code.
- Managing distributed work, dependencies, and results can become complicated.
- You want to scale your existing Python workflow without starting over.

Dask helps you cross these boundaries.
Same Python ecosystem. Larger data. More cores. Multiple machines.

What is Dask?

A parallel computing library in Python that allows you to scale computations from a single machine to a distributed cluster. It is particularly useful when:

1. Your data does not fit in memory
2. Your code is CPU-bound and slow
3. You want to parallelize Python workflows without rewriting everything

Dask Distributed Cluster

To perform work, a scheduler must be assigned resources in the form of a Dask cluster. The Dask cluster has three main components for processing computations in parallel.

YOU

Client

The client is responsible for submitting tasks to be executed to the scheduler. It also enables you to monitor job progress and access the dashboard.

CONDUCTOR

Scheduler

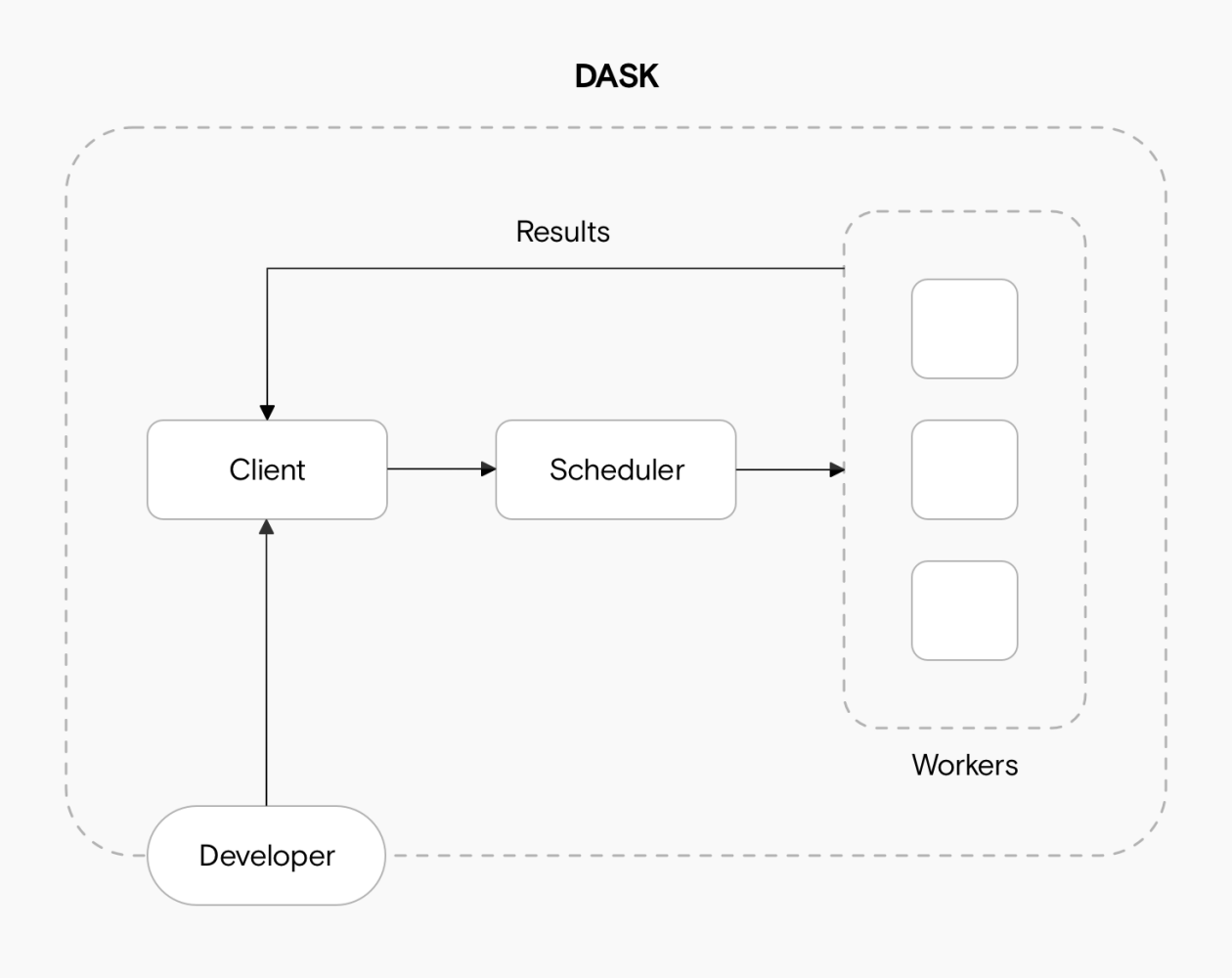
Decides how work is split across workers and coordinates the parallelized workload.

THE ORCHESTRA

Workers

The workers compute tasks and store and return computations results. Workers can be threads, processes, or separate machines in a cluster.

What that looks like



Launch a local cluster from Open OnDemand

```
from dask.distributed import LocalCluster,
Client

cluster = LocalCluster(
    n_workers=4,
    threads_per_worker=2,
    memory_limit='auto',
)
client = Client(cluster)
client   # dashboard link lives here
```

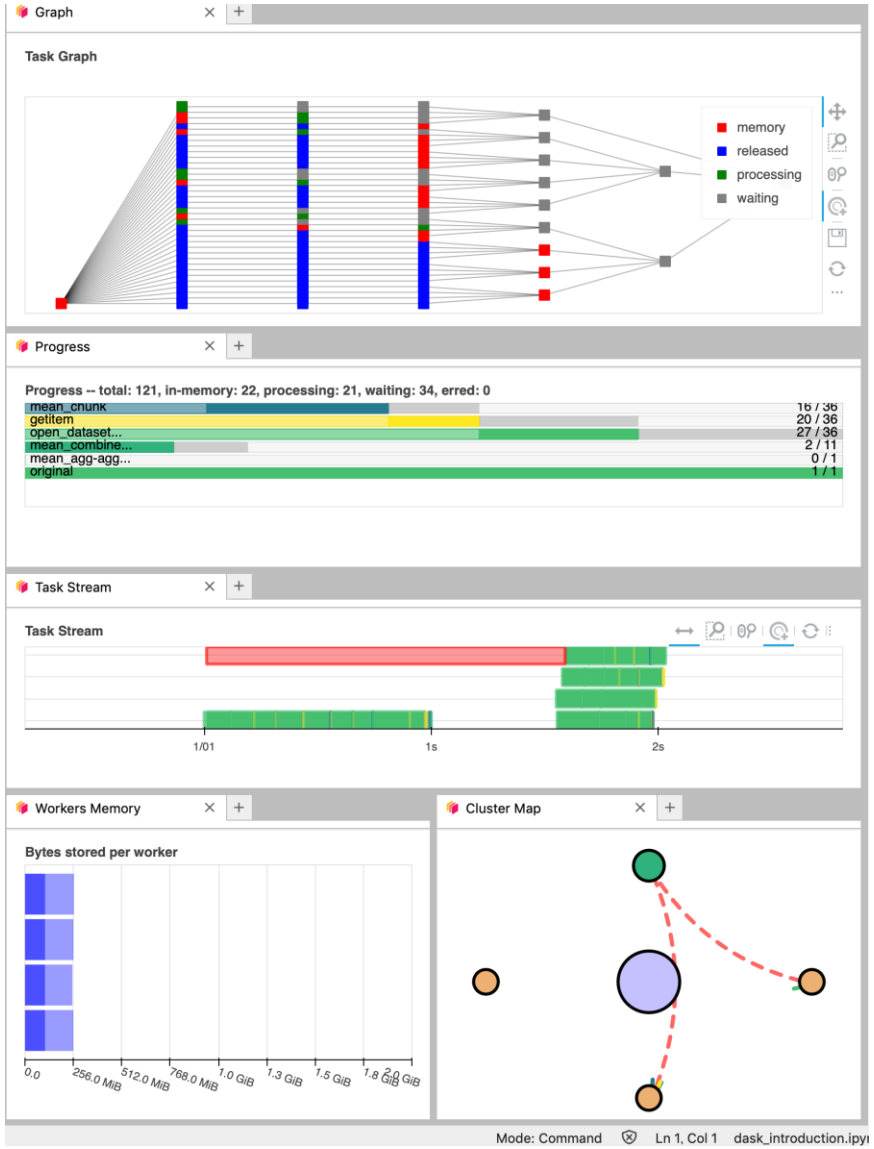
- A LocalCluster runs all components on a single node and is useful for development and small-scale parallel workloads.
- For this we need to set up a LocalCluster using dask.distributed and connect a client to it.

Things to note:

- Match workers × threads to the cores you requested
- No arguments? Dask sizes itself to the session
- When you are done: client.close() and cluster.close()

The dashboard is your X-ray

Dask Dashboard is a web interface that provides real-time insights into task execution, CPU and memory usage, and worker activity.



WATCH FOR:

- Task stream: are many workers busy at once?
- Progress: which functions are stuck?
- Worker memory: is one worker stockpiling?
- Graph: did you accidentally build one task?

Caveat: the proxy URL

REWRITE THE URL

Dask reports this and it will not open in OOD:

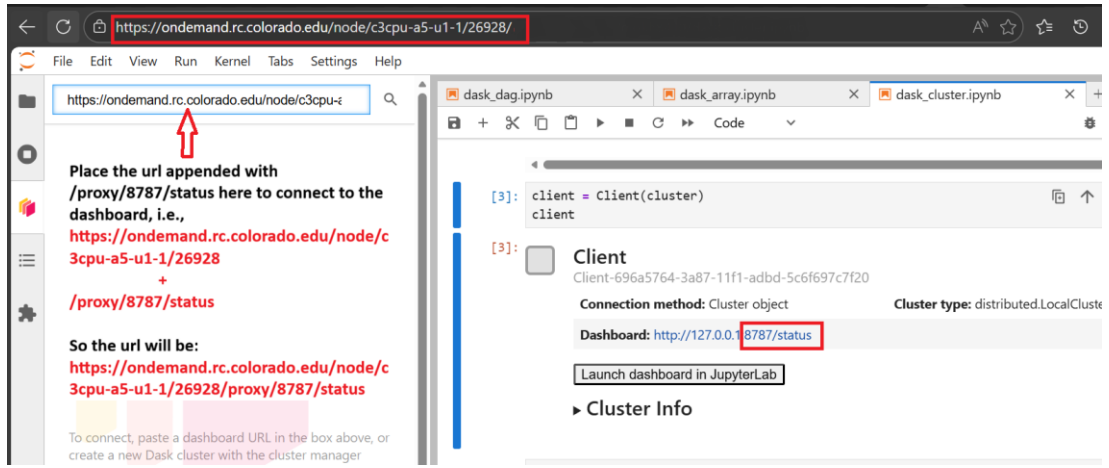
```
http://127.0.0.1:8787/status
```

Your Jupyter session looks like this:

```
https://ondemand.rc.colorado.edu/node/<node>/<port>/
```

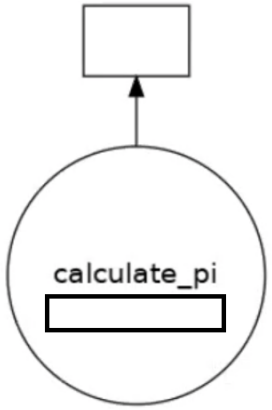
Append the proxy path:

```
https://ondemand.rc.colorado.edu/node/<node>/<port>/proxy/8787/status
```

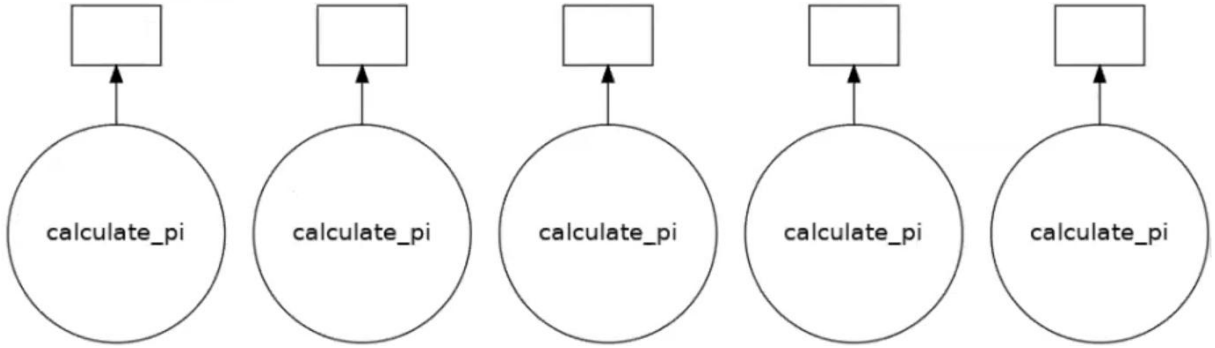


Visualizing the Task Graph

One call = one task. Nothing to parallelize.

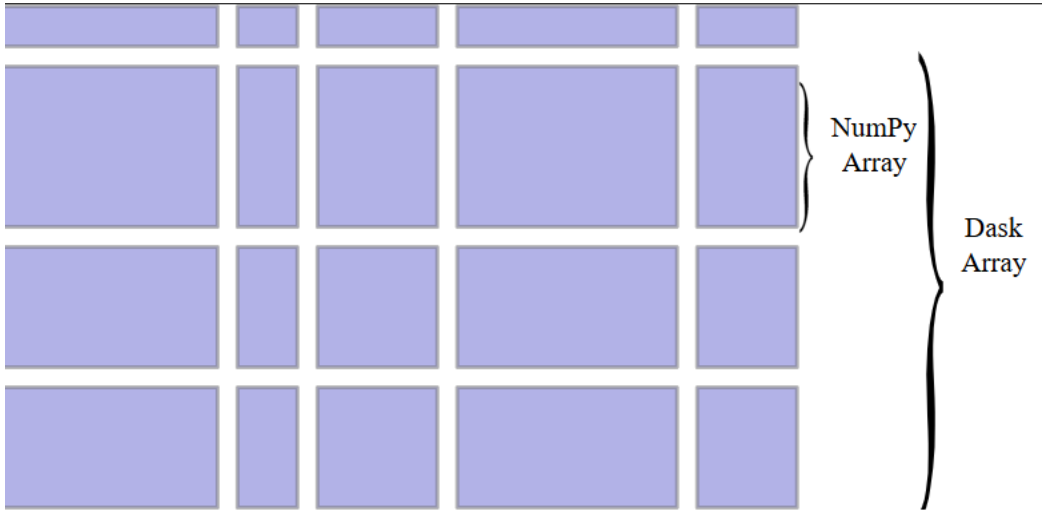


Five independent calls = five workers can go at once.




```
import dask
results = [dask.delayed(calculate_pi)(10000 * (i + 1)) for i in range(5)]
dask.visualize(results)     # inspect the graph first
dask.compute(*results)      # then run it
```

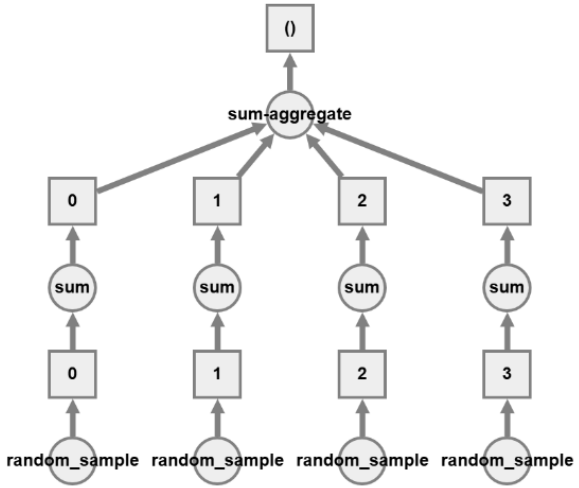
Dask Arrays: NumPy, in chunks



One Dask Array = many NumPy chunks

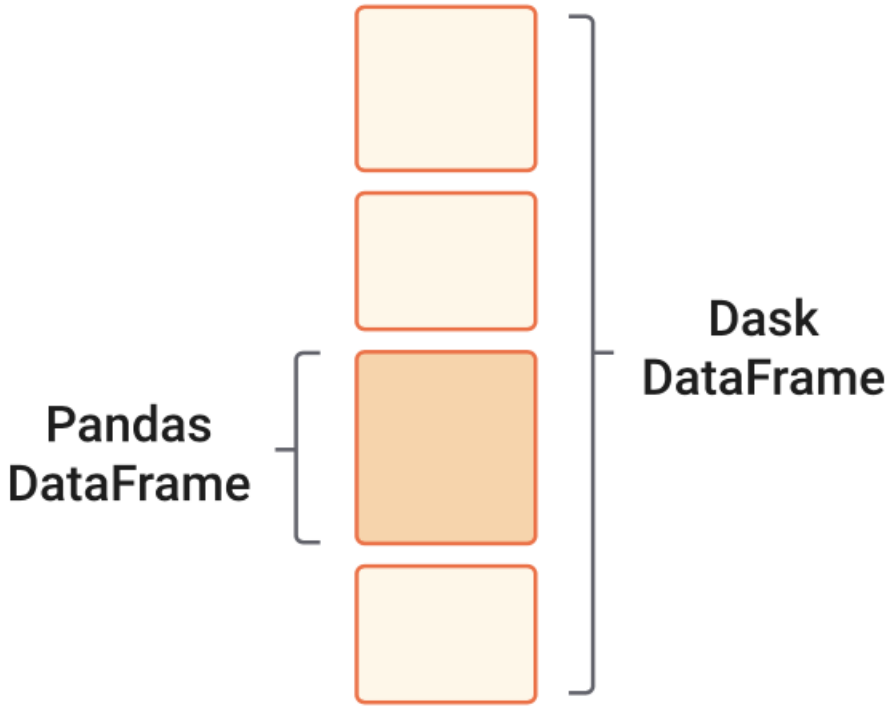
```
import dask.array as da

x = da.random.random(
    (10000, 10000), chunks=(1000, 1000)
)
y = (x + x.T).mean()   # still lazy
y.compute()            # now it runs
```



- Works like NumPy arrays but can handle data larger than memory.
- Splits large arrays into smaller chunks and processes them in parallel.
- Operations on different chunks run simultaneously across CPU cores or workers.

Dask DataFrames: Pandas, in partitions




```
import dask.dataframe as dd

df = dd.read_csv('data/file.csv')   # lazy
filtered = df[df['column'] > 10]
grouped = filtered.groupby('category').mean()

result = grouped.compute()   # pandas
DataFrame
```

- Splits a large DataFrame into smaller Pandas DataFrames called partitions.
- Follows lazy computations, so it delays computations until needed, allowing Dask to optimize the workflow.

Demo ·

1. **Open OnDemand**
   Jupyter session on Alpine with enough cores to see parallelism.

2. **LocalCluster + Client**
   Start workers, then connect. Confirm total threads.

3. **Fix the dashboard URL**
   Session URL + /proxy/8787/status. Then open Task Stream.

4. **Compute an array**
   Dask Array (x + x.T).mean(), and watch chunks run in parallel.

5. **Close the client**
   client.shutdown() so workers and memory are released.

Any Questions